Monad Execution

Security Assessment

March 20th, 2026 — Prepared by OtterSec

James Wang james.wang@osec.io

Jinwoo Lee jin@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MOE-ADV-00 | Premature Warm Access Accounting 7

OS-MOE-ADV-01 | Undefined Behavior on Null memcpy 8

OS-MOE-ADV-02 | Misaligned Base Fee Eviction Policy 9

OS-MOE-ADV-03 | Insufficient Defensive RLP Validation 10

General Findings **11**

OS-MOE-SUG-00 | Redundant Write After Pop Removal 12

OS-MOE-SUG-01 | Missing Empty Input Validation 13

OS-MOE-SUG-02 | Unchecked ABI Head Offsets 14

OS-MOE-SUG-03 | Discrepancies between Documentation and Implementation 15

Vulnerability Rating Scale **17**

Procedure **18**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Category Labs engaged OtterSec to assess the `execution` program. This assessment was conducted between January 19th and February 6th, 2026. For more information on our auditing methodology, refer to chapter 07.

Key Findings

We produced 8 findings throughout this audit engagement.

In particular, we identified a vulnerability where the interpreter may mark an account warm by calling the access-account method before verifying that enough gas remains for the cold-access cost. This may enable attackers trigger disproportionately expensive cold-account work from frames funded closer to the warm-cost path, potentially threatening block execution within Monad's block voting time window (OS-MOE-ADV-00). Another issue concerns calling memcpy with a null source pointer is undefined behavior, even if the copy length is zero, and may potentially result in correctness and safety issues (OS-MOE-ADV-01).

We also made advised updating the validator addition logic to incorporate explicit checks to ensure the head offsets conform to the expected ABI layout, providing stricter input validation (OS-MOE-SUG-02). We further highlighted several instances where the code implementation diverges from the documented behavior resulting in ambiguity, and suggested aligning the implementations with the expected behavior stated in the documentation (OS-MOE-SUG-03).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a git repository at https://github.com/category-labs/monad. This audit was performed against commit c54fc3a.

A brief description of the programs is as follows:

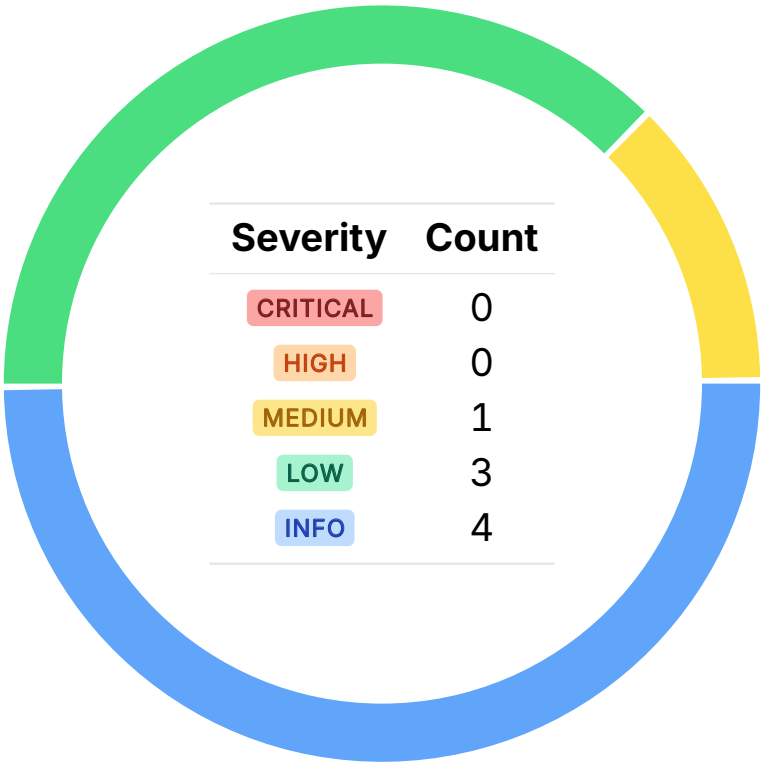
Name	Description
execution	It is the chain's state-transition engine plus EVM runtime/compiler stack, extended with Monad-specific precompiles and system logic such as staking and epoch/reward processing.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 8 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	3
INFO	4

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

ID	Severity	Status	Description
OS-MOE-ADV-00	MEDIUM	RESOLVED	The interpreter may mark an account warm by calling `access_account` before verifying that enough gas remains for the cold-access cost. This may enable attackers trigger disproportionately expensive cold-account work from frames funded closer to the warm-cost path, potentially threatening block execution within Monad's block voting time window.
OS-MOE-ADV-01	LOW	RESOLVED	Calling `std::memcpy` with a null source pointer is undefined behavior, even if the copy length is zero, and may potentially result in correctness and safety issues.
OS-MOE-ADV-02	LOW	RESOLVED	The `txpool` evicts transactions utilizing a fixed `MIN_BASE_FEE`, but block proposal utilizes the current dynamic base fee, so underpriced transactions may remain in the pool even though they cannot currently be included, wasting pool capacity and resulting in builders to repeatedly skip stale transactions during proposal construction.
OS-MOE-ADV-03	LOW	RESOLVED	The RLP decoder may be vulnerable to `size_t` overflow when computing slice endpoints, allowing malformed lengths to bypass bounds checks, and the EIP-4844 path appears to accept unconsumed trailing payload bytes.

© 2026 Otter Audits LLC. All Rights Reserved.

© 2026 Otter Audits LLC. All Rights Reserved.

Premature Warm Access Accounting MEDIUM OS-MOE-ADV-00

Description

The interpreter determines cold/warm status by calling `ctx->host->access_account` before it has established that enough gas remains to pay the full cost of the operation. This implies the host may perform the access and mark the account warm before gas validation for the full cold-cost path is complete. An attacker may exploit this by entering through a warm external call funded near the warm-cost minimum, then execute an opcode such as `EXTCODESIZE` on a cold account. If the cold path is triggered after the access-state mutation, the runtime may perform much more expensive host-side work than the frame's gas budget was intended to permit. This mismatch may create disproportionate runtime cost relative to the gas budget that was supposed to constrain it, increasing execution latency enough to threaten block processing within the voting window.

>_ *category/vm/runtime/data.hpp* cpp

```cpp
template <Traits traits>
void extcodecopy(
    Context *ctx, uint256_t const *address_ptr,
    uint256_t const *dest_offset_ptr, uint256_t const *offset_ptr,
    uint256_t const *size_ptr)
{
    [...]
    if constexpr (traits::eip_2929_active()) {
        auto const access_status =
            ctx->host->access_account(ctx->context, &address);
        if (access_status == EVMC_ACCESS_COLD) {
            ctx->deduct_gas(traits::cold_account_cost());
        }
    }
    [..]
}
```

Remediation

Redesign the storage access to ensure checks can happen before costly operations.

Patch

The Monad team acknowledges the risk and remediation approach. Given current priorities, the team considers the residual risk acceptable at this time and will continue to monitor and revisit the recommended changes.

© 2026 Otter Audits LLC. All Rights Reserved.

Undefined Behavior on Null memcpy LOW OS-MOE-ADV-01

Description

`std::memcpy` requires both source and destination pointers to be valid for the operation being formed. If `input_data` is `nullptr` and is passed directly as the source pointer, then the call is undefined behavior once the implementation treats that pointer as an operand to `memcpy`. In C++, this is not automatically made safe just because the byte count happens to be zero. Some implementations may appear to tolerate `memcpy(..., nullptr, 0)`, but optimizers and `libc` are allowed to assume the pointer preconditions are satisfied. The risk is more serious if `input_data` is not guaranteed non-null when `size > 0`, as the call may attempt to read from an invalid source address, resulting in correctness or memory-safety issues.

Independently, in `copy_impl`, `source == nullptr` can reach a `std::copy_n` invocation without sufficient validation.

Remediation

Explicitly check that `input_data != nullptr` whenever the copy is reachable, or guard the call so `memcpy` is only invoked when the size is nonzero and the source pointer is known valid.

Patch

The Monad team acknowledges the risk and remediation approach. Given current priorities, the team considers the residual risk acceptable at this time and will continue to monitor and revisit the recommended changes.

© 2026 Otter Audits LLC. All Rights Reserved.

Misaligned Base Fee Eviction Policy LOW OS-MOE-ADV-02

Description

Eviction and proposal selection are utilizing different fee baselines. Eviction ranks transactions against a fixed `MIN_BASE_FEE`, while proposal construction tests them against the live base fee. When the live base fee rises, some transactions become non-includable for current blocks but may still score well enough under the lower fixed baseline to survive pool eviction. This creates a class of transactions that are too high-priority to be evicted by the txpool but too low-priced to be included by the proposer. As a result, such transactions may persist in the pool under pressure and be repeatedly skipped during proposal construction, wasting pool capacity and potentially displacing transactions that are closer to inclusion.

Remediation

Align eviction scoring with the same dynamic fee basis utilized for inclusion.

Patch

The Monad team acknowledges the risk and remediation approach. Given current priorities, the team considers the residual risk acceptable at this time and will continue to monitor and revisit the recommended changes.

© 2026 Otter Audits LLC. All Rights Reserved.

Insufficient Defensive RLP Validation LOW OS-MOE-ADV-03

Description

the RLP decoder appears to trust the encoded structure more than a defensive parser normally would. The current design assumes that all encoded data is produced by the local consensus daemon. It is unclear whether RLP decoding is treated as a security boundary or merely as an internal invariant. In the RLP decoder, the calculation of `end` may overflow before the check (`end > enc.size`) is performed. If an overflow occurs, `end` may wrap to a smaller value before the comparison, so the bounds check may be bypassed even though the requested slice actually extends past the buffer. This may result in malformed parsing, where a `substr` call could throw an exception or cause downstream invalid memory access.

Separately, in the EIP-4844 transaction handling path, `hashes_payload` is not fully consumed or validated after decoding the expected fields, implying trailing bytes may remain silently attached to an otherwise accepted transaction object. Even if EIP-4844 is not currently active, this weakens canonical format enforcement, as incomplete payload validation may allow arbitrary data to be embedded in serialized transaction or proposal structures.

Remediation

Validate the lengths before summing the values when calculating `end` . Additionally, in EIP-4844 transaction, require full consumption of the payload and reject any leftover bytes.

Patch

The Monad team acknowledges the risk and remediation approach. Given current priorities, the team considers the residual risk acceptable at this time and will continue to monitor and revisit the recommended changes.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MOE-SUG-00	The pop-and-swap removal path always writes `swapped_id` back into the removed slot, even when that slot was already the last live array entry, resulting in unnecessary write outside the logical active range of the array.
OS-MOE-SUG-01	`get_epoch` and `get_proposer_val_id` do not enforce empty `calldata`, so they accept arbitrary trailing input despite being zero-argument getters.
OS-MOE-SUG-02	`precompile_add_validator` ignores the ABI `head` offsets for its three dynamic bytes arguments, so malformed offset values are silently accepted as long as the `tail` data is in the expected order.
OS-MOE-SUG-03	There are several instances where the code implementation diverges from the documented behavior resulting in ambiguity.

© 2026 Otter Audits LLC. All Rights Reserved.

Redundant Write After Pop Removal

OS-MOE-SUG-00

Description

In `staking_contract::syscall_snapshot`, in the pop-and-swap removal logic, `vars.valset_execution.p` first shortens the logical array by removing the last element, and then `slot_to_replace.store(swapped_id)` writes the popped value into the slot being removed. If the removed element was already the last array entry, `pop` reduces the logical length first, so that subsequent store writes into a slot that now lies beyond the active array bounds.

>_ *category/execution/monad/staking/staking_contract.cpp* cpp

```cpp
Result<void> StakingContract::syscall_snapshot(
    byte_string_view const input, uint256_t const &value)
{
    [...]
    // 6. Process removals from execution set to prevent state bloat.
    // Pop-and-swap from the array: highest indices must processed first.
    for (auto it = removals.rbegin(); it != removals.rend(); ++it) {
        auto slot_to_replace = vars.valset_execution.get(*it);
        u64_be const id_to_remove = slot_to_replace.load();
        remove_from_valset(id_to_remove);
        auto const swapped_id = vars.valset_execution.pop();
        slot_to_replace.store(swapped_id);
    }

    vars.in_epoch_delay_period.store(true);
    return outcome::success();
}
```

Remediation

Perform `slot_to_replace.store(swapped_id)` only when the removal index is different from the old last index.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Empty Input Validation

OS-MOE-SUG-01

Description

`get_epoch` and `get_proposer_val_id` are zero-argument getters, but they never verify that input is empty. As a result, callers may append arbitrary trailing `calldata` and still get a successful response, rendering the interface more permissive than the declared ABI.

>_ *category/execution/monad/staking/staking_contract.cpp* cpp

```cpp
Result<byte_string> StakingContract::precompile_get_epoch(
    byte_string_view const, evmc_address const &,
    evmc_uint256be const &msg_value)
{
    BOOST_OUTCOME_TRY(function_not_payable(msg_value));

    AbiEncoder encoder;
    encoder.add_uint(vars.epoch.load());
    encoder.add_bool(vars.in_epoch_delay_period.load());
    return encoder.encode_final();
}

Result<byte_string> StakingContract::precompile_get_proposer_val_id(
    byte_string_view, evmc_address const &, evmc_uint256be const &msg_value)
{
    BOOST_OUTCOME_TRY(function_not_payable(msg_value));

    AbiEncoder encoder;
    encoder.add_uint(vars.proposer_val_id.load());
    return encoder.encode_final();
}
```

Remediation

Add an explicit empty-input check to ensure these getters are consistent with the rest of the precompile interface.

© 2026 Otter Audits LLC. All Rights Reserved.

Unchecked ABI Head Offsets

OS-MOE-SUG-02

Description

`add_validator` reads the three ABI `head` words but only discards them, so it never verifies that the dynamic offsets actually point to the expected tail locations. As a result, callers may supply arbitrary offset values for the offset. However there is no visible impact from this sicne the message and signatures are still decoded and verified correctly from the `tail`.

>_ *category/execution/monad/staking/staking_contract.cpp* RUST

```rust
template <Traits traits>
Result<byte_string> StakingContract::precompile_add_validator(
    byte_string_view input, evmc_address const &,
    evmc_uint256be const &msg_value)
{
    constexpr size_t MESSAGE_SIZE = 33 /* compressed secp pubkey */ +
                                    48 /* compressed bls pubkey */ +
                                    sizeof(Address) /* auth address */ +
                                    sizeof(u256_be) /* signed stake */ +
                                    sizeof(u256_be) /* commission rate */;
    // decode the head
    BOOST_OUTCOME_TRY(
        abi_decode_fixed<u256_be>(input)); // skip message tail offset
    [...]
}
```

Remediation

Add explicit checks to ensure the `head` offsets conform to the expected ABI layout, providing stricter input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

Discrepancies between Documentation and Implementation OS-MOE-SUG-03

Description

1. In `compound` the current flow calls `pull_delegator_up_to_date`, reads `del.rewards`, clears it, and only redelegates if rewards are nonzero. If the delegator record is absent, the call silently succeeds without reverting, going against the documentation, which explicitly states that the call should revert when no delegator account exists. Also, the function currently emits `ClaimRewards` even though the documentation says that event is emitted only when rewards are claimed via `claimRewards`.

>_ *category/execution/monad/staking/staking_contract.cpp* RUST

```rust
template <Traits traits>
Result<byte_string> StakingContract::precompile_compound(
    byte_string_view input, evmc_address const &msg_sender,
    evmc_uint256be const &msg_value)
{
    [...]
    auto del = vars.delegator(val_id, msg_sender);
    BOOST_OUTCOME_TRY(pull_delegator_up_to_date(val_id, del));
    auto rewards_slot = del.rewards();
    uint256_t const rewards = rewards_slot.load().native();
    rewards_slot.clear();

    if (MONAD_UNLIKELY(rewards != 0)) {
        [...]
        emit_claim_rewards_event(val_id, msg_sender, rewards);
        BOOST_OUTCOME_TRY(delegate<traits>(val_id, rewards, msg_sender));
    }
    return byte_string{abi_encode_bool(true)};
}
```

2. `claimRewards` loads the delegator, updates it, reads `del.rewards`, and transfers tokens only if the reward balance is nonzero. Thus it loads the delegator slot directly and continue execution without first verifying that a delegator record exists. If the delegator does not exist, it succeeds as harmless no-ops instead of reverting. This behavior contradicts the documented contract semantics.

>_ *category/execution/monad/staking/staking_contract.cpp* RUST

```rust
Result<byte_string> StakingContract::precompile_claim_rewards(
    byte_string_view input, evmc_address const &msg_sender,
    evmc_uint256be const &msg_value)
{
    [...]
    auto const rewards = del.rewards().load();
    if (MONAD_LIKELY(rewards.native() != 0)) {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        send_tokens(msg_sender, rewards.native());
        del.rewards().clear();
        emit_claim_rewards_event(val_id, msg_sender, rewards);
    }

    return byte_string{abi_encode_bool(true)};
}
```

3. `externalReward` currently emits `ValidatorRewarded` even though the documentation states that it is emitted when block reward is allocated via `syscallReward`. Also, the implementation derives the minimum allowed external reward from `min_external_reward`, which currently returns `dust_threshold`. This behavior does not align with the documented value for `externalReward`.

>_ *category/execution/monad/staking/staking_contract.cpp* RUST

```
Result<byte_string> StakingContract::precompile_external_reward(
    byte_string_view input, evmc_address const &sender,
    evmc_uint256be const &msg_value)
{
    [...]
    // 3. Update validator accumulator.
    BOOST_OUTCOME_TRY(
        apply_reward(val_id, sender, external_reward, active_stake));
    return byte_string{abi_encode_bool(true)};
}
```

4. `delegate` currently treats `msg.value == 0` as a successful no-op and returns true, while the documentation says a zero-amount delegation should revert.

>_ *category/execution/monad/staking/staking_contract.cpp* RUST

```
template <Traits traits>
Result<byte_string> StakingContract::precompile_delegate(
    byte_string_view input, evmc_address const &msg_sender,
    evmc_uint256be const &msg_value)
{
    [...]
    if (MONAD_LIKELY(stake != 0)) {
        BOOST_OUTCOME_TRY(delegate<traits>(val_id, stake, msg_sender));
    }
    return byte_string{abi_encode_bool(true)};
}
```

Remediation

Ensure the the implementations align with the expected behavior stated in the documentation.

© 2026 Otter Audits LLC. All Rights Reserved.

06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.